

Generics

Java Advanced Developer

StayAhead Training; Aug, 2025

About Generics

Generics

- Code that can handle a variety of types whilst remaining type safe
- What's wrong with this code?

```
var course = new ArrayList();  
course.add("Java Advanced Developer");  
course.add(5);  
course.add(LocalDate.of(2025, 8, 4));
```

Generic Types

Generics

- A class/interface with one or more type parameters
- A class that requires type info at instantiation time
- An interface that requires type info at implementation time
- Type parameter: like a method parameter but for which the arg. is a type

Generic Types

Generics

```
class Container<T> {  
    T contents;  
  
    void setContents(T contents) {  
        this.contents = contents;  
    }  
  
    T getContents() {  
        return this.contents;  
    }  
}
```



Generic Types

Generics

- The type argument(s) may be inferred

```
Container<String> container = new Container<String>();
```

```
Container<String> container = new Container<>();
```

```
var container = new Container<String>();
```

Generic Types

Generics

```
interface Transformer<T, R> {  
    R transform(T t);  
}
```

Multiple type parameters

Generic Types

Generics

```
class MyTransformer implements Transformer<String, Integer> {  
    @Override  
    public Integer transform(String t) {...}  
}
```

// or...

```
class MyTransformer<T, R> implements Transformer<T, R> {  
    @Override  
    public R transform(T t) {...}  
}
```

Generic Types: Type Parameter Naming

Generics

- Type parameter names conventionally one upper case letter, e.g.:
 - T for type
 - R for return value
 - E for element
 - K for key
 - V for value

Raw Types

Generics

- A generic class/interface without type arguments

```
var container = new Container();
```

- The type argument is assumed to be `Object`
- You can assign a parameterised type to a raw type & vice-versa but don't!
- **Avoid raw types**

Generic Methods

Generics

- Methods in a non-generic class/interface may be generic

The type parameter

```
public <T> T echo(T t) {  
    System.out.println(t);  
    return t;  
}
```

Generic Methods

Generics

- Calling a generic method with explicit type argument (type witness):

```
var result = someObject.<String>echo("Hello");
```

- The type argument can be inferred from the method argument(s)

```
var result = someObject.echo("Hello");
```


Type Inference

Generics

- The ability of the compiler to determine the type arg(s) from the context, e.g.:

```
Container<String> container = new Container<>();
```

```
var result = someObject.echo("Hello");
```

No type arg.

- Where the generic method accepts no args the target type is used to infer

```
String result = someObject.someGenericMethod();
```

Bounded Type Parameters

Generics

- A type parameter that is limited to certain types, e.g.:

```
class Basket<T extends Chargeable> {...}
```

- Only Chargeable or one of its sub-types is applicable (upper bounded)
- Note that `extends` covers both `extends` and `implements`
- Use the binary operator `&` to apply multiple bounds, e.g. `A & B & C`

Inheritance

Generics

- This is ok:

```
void processNumber(Number number) {  
    ...  
}
```

```
// ---
```

```
Integer num = 42;  
someObject.processNumber(num);
```



Inheritance

Generics

- But this is not:

```
void processNumbers(List<Number> numbers) {  
    ...  
}
```

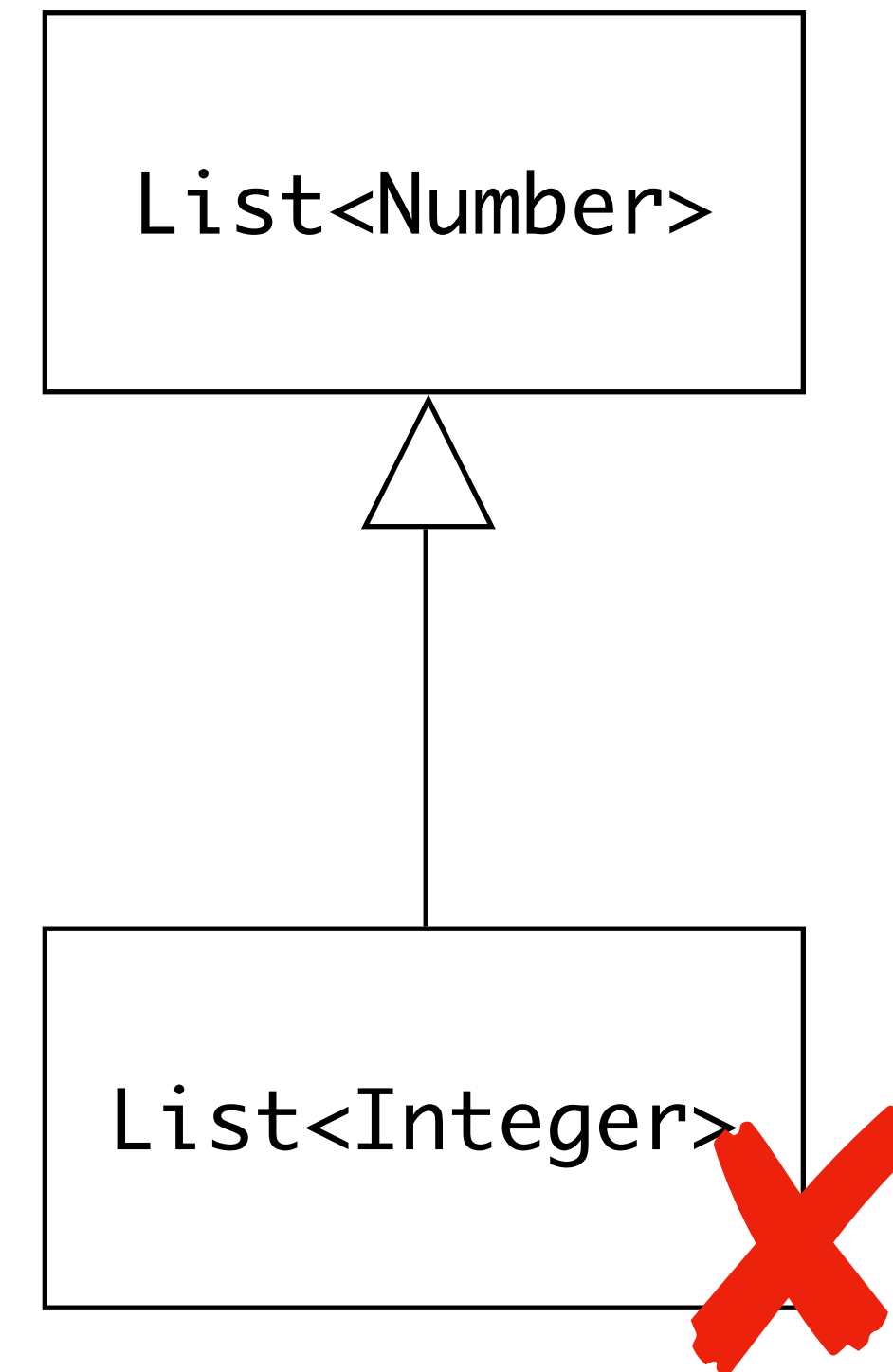
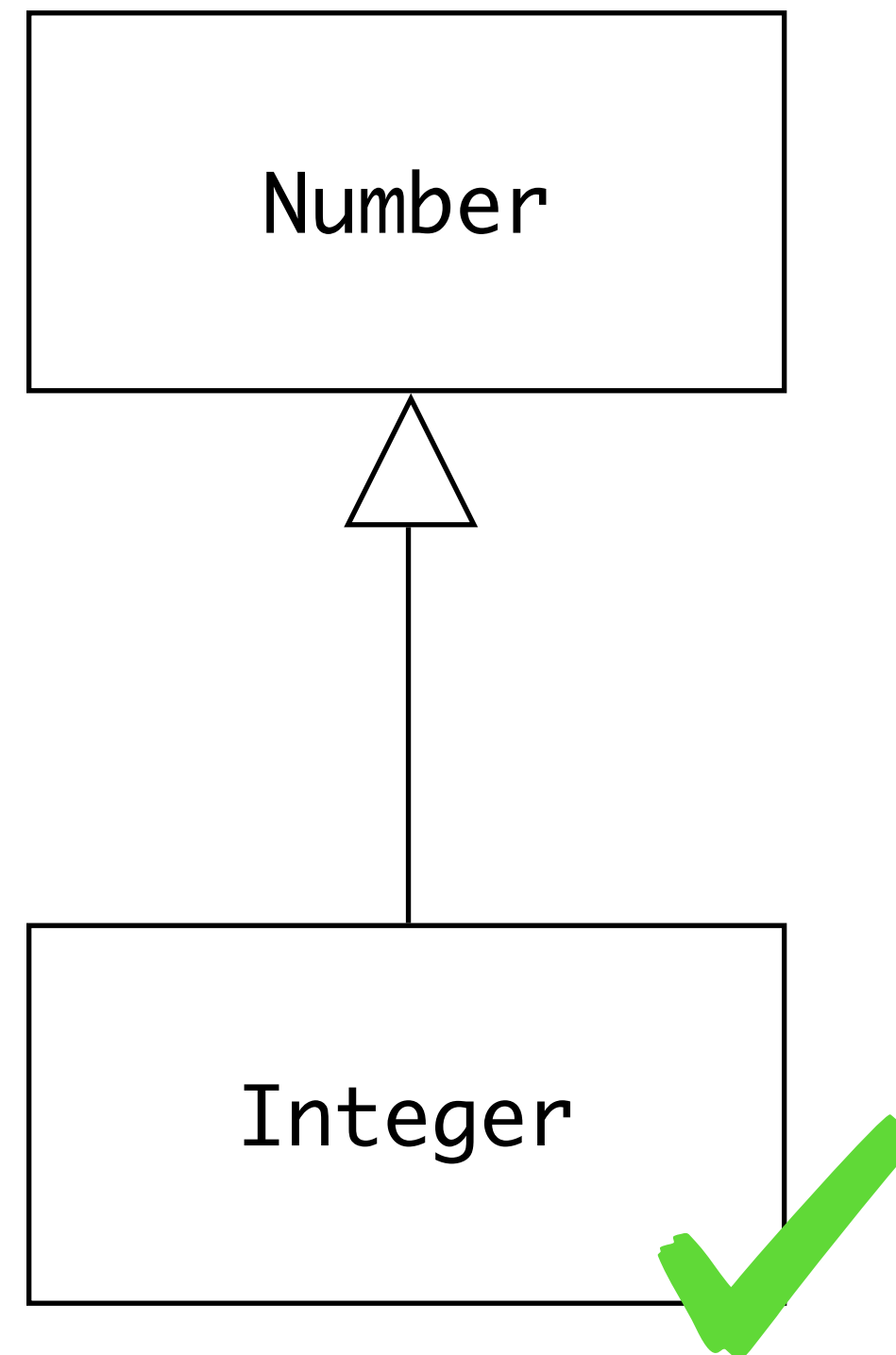
```
// ---
```

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
someObject.processNumbers(nums);
```



Inheritance

Generics



Inheritance

Generics

- Imagine it were possible:

```
void processNumbers(List<Number> numbers) {  
    numbers.add(4.5);  
}
```

nums now contains a
value that is NOT
an Integer!

```
// ---
```

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
someObject.processNumbers(nums);
```

Inheritance

Generics

- Upper bounded type parameter to solve the inheritance problem:

```
<T extends Number> void processNumbers(List<T> numbers) {  
    ...  
}
```

```
// ---
```

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
someObject.processNumbers(nums);
```



Wildcards

Generics

- The wildcard ? is an alternative to the type parameter in this case:

```
void processNumbers(List<? extends Number> numbers) {  
    ...  
}
```

```
// ---
```

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
someObject.processNumbers(nums);
```



Wildcards: Wildcard or Type Parameter?

Generics

- Wildcard code is arguably simpler
- Wildcard may be lower bounded or unbounded
- Code can be unwieldy when wildcard is used more than once in a method
- Type parameters provide for dependencies between type arguments, e.g.:
 - if the type of arg2 is dependent on arg1
 - if the return type is dependent on the arg type

Unbounded Wildcard

Generics

- What's wrong with this code?

```
void saveToFile(List<Object> list, Path path) throws... {  
    for (var element : list) {  
        Files.writeString(path, element.toString());  
    }  
}
```

Unbounded Wildcard

Generics

- A List of Books is not a List of Objects

```
var books = new ArrayList<Book>();
```

...

```
someObject.saveToFile(books, path);
```

Compilation error

Unbounded Wildcard

Generics

- Use the unbounded wildcard when the code depends on Object only

```
void saveToFile(List<?> list, Path path) throws... {  
    for (var element : list) {  
        Files.writeString(path, element.toString());  
    }  
}
```

Lower Bounded Wildcard

Generics

- Limit the type to the specified type or one of its super-types, e.g.:

`List<? super IOException>`

- Only `IOException`, `Exception`, `Throwable`, or `Object` is applicable

Covariance

Generics

- `List<? extends Number>`
- Is not the same as `List<Number>`
- The type argument may be `List<Number>` or `List<Double>` etc.
- The code in the method may not add anything to the `List` except `null`...
- ...because, e.g. you cannot put a `double` into a `List<Integer>`

Contravariance

Generics

- `List<? super IOException>`
- Is not the same as `List<IOException>`
- The type argument may be `List<IOException>` or `List<Exception>`
- The code in the method may add `IOException` to the `List...`
- ...because, e.g. you can put an `IOException` in a `List<Exception>`

PECS

Generics

- Producer **extends** consumer **super**
- If the generic object, e.g. `List`, produces data use upper bounded wildcard

`List<? extends Number> producerList`

- If the generic object, e.g. `List`, consumes data use lower bounded wildcard

`List<? super Number> consumerList`

Type Erasure

Generics

- The replacement of type parameters with:
 - `Object` if unbounded
 - the upper/lower bound if bounded...by the **compiler**
- Note that type arguments are erased too

Type Erasure: From Generics

```
class Container<T> {  
    T contents;  
  
    void setContents(T contents) {  
        this.contents = contents;  
    }  
  
    T getContents() {  
        return contents;  
    }  
}
```

Type Erasure: To Generics

```
class Container {  
    Object contents;  
  
    void setContents(Object contents) {  
        this.contents = contents;  
    }  
  
    Object getContents() {  
        return contents;  
    }  
}
```


Type Erasure: From Generics

```
<T extends Number> T getBiggest(T[] nums) {  
    T biggest = nums[0];  
  
    for (var num : nums) {  
        if (num.doubleValue() > biggest.doubleValue()) {  
            biggest = num;  
        }  
    }  
  
    return biggest;  
}
```

Type Erasure: To Generics

```
Number getBiggest(Number[] nums) {  
    Number biggest = nums[0];  
  
    for (var num : nums) {  
        if (num.doubleValue() > biggest.doubleValue()) {  
            biggest = num;  
        }  
    }  
  
    return biggest;  
}
```


Restrictions

Generics

- Cannot instantiate a generic type with primitive types, e.g.:

```
var container = new Container<int>();
```



- Cannot create an instance of a type parameter, e.g.:

```
var t = new T();
```



- Cannot declare static fields whose types are type parameters, e.g.:

```
private static T t;
```

 `// all instances share the field!`

Restrictions

Generics

- Cannot use `instanceof` with a parameterised type, e.g.:

```
if (nums instanceof List<Integer>) {...}
```



- Cannot cast to a parameterised type (some exceptions), e.g.:

```
var ints = (List<Integer>) nums;
```



- Generic types cannot extend `Throwable`, e.g.:

```
class MyException<T> extends Exception {...}
```



Restrictions

Generics

- Cannot throw or catch a parameterised type, e.g.:

```
catch (T t) {...}
```



- Cannot have n overloads with the same signature after type erasure, e.g.:

```
void saveToFile(List<Book> books) {...}
```

```
void saveToFile(List<Review> reviews) {...}
```

